

# NexusAI

## Full Product Implementation Specification

Knowledge Base • Lead Generation • WhatsApp Growth • Analytics • Integrations

*Version 2.0 — March 2026 — For Claude Code*

| Field         | Details                                                                                         |
|---------------|-------------------------------------------------------------------------------------------------|
| Repo          | github.com/tech-hive-dev/nexusai                                                                |
| Stack         | FastAPI · Next.js 14 · PostgreSQL + pgvector · Claude API · OpenAI Embeddings                   |
| Channels      | Website widget · WhatsApp Business API · Facebook Messenger                                     |
| Target Market | Small & Medium Enterprises (SMEs) — SaaS multi-tenant platform                                  |
| Scope v2.0    | Knowledge Base + Lead Gen + WhatsApp Automation + Analytics + Industry Templates + Integrations |

# 1. Product Vision & Feature Map

NexusAI targets SMEs who need to acquire new customers and retain existing ones — without hiring extra staff. The platform must cover three customer lifecycle stages:

| ATTRACT — Win New Customers                                                                            | SERVE — Excellent Support                                                                                 | RETAIN — Keep & Grow                                                                       |
|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| AI Lead Qualification<br>Instant Quote Generator<br>Appointment Booking<br>WhatsApp Proactive Outreach | 24/7 Intelligent Responses<br>Auto-Growing Knowledge Base<br>Sentiment Escalation<br>Multilingual Support | Post-Sale Check-ins<br>Google Review Automation<br>Referral Program<br>WhatsApp Broadcasts |

## 2. Knowledge Base Enhancement (v1 Summary)

The following was specified in v1 of this document and should be implemented first as the foundation for all other features. Key deliverables:

- `web_crawler.py` — crawl URLs, ingest to pgvector
- `document_parser.py` — PDF, DOCX, XLSX, PPTX, CSV via unstructured library
- `knowledge_gap_filler.py` — DuckDuckGo search when agent confidence < 0.65
- `crm_connector.py` — HubSpot and Salesforce sync
- `knowledge_refresh.py` — APScheduler 24-hour re-crawl worker
- `knowledge_sources` API — REST endpoints for managing all sources

### Prerequisite

Complete all items in Section 2 before proceeding to Sections 3–10. The lead qualification, analytics, and automation features all depend on a working knowledge base and RAG pipeline.

## 3. AI Lead Qualification Engine

### 3.1 Overview

When a visitor chats on the website or WhatsApp, the bot proactively collects qualification signals — budget, timeline, decision-maker status, specific need — and assigns a lead score. High-scoring leads are immediately flagged to the SME owner via WhatsApp or email notification.

### 3.2 New File: `services/lead_qualifier.py`

```
from anthropic import Anthropic
from app.models.lead import Lead, LeadScore
from app.core.database import get_db
import json

client = Anthropic()

QUALIFICATION_PROMPT = """
You are a lead qualification assistant. Based on the conversation below,
extract and score this lead on a scale of 0-100.

Scoring criteria:
- Budget confirmed or implied (0-25 pts)
- Clear timeline / urgency (0-25 pts)
- Decision maker (0-25 pts)
- Specific need matches our service (0-25 pts)

Return JSON: {"score": int, "budget": str, "timeline": str,
"decision_maker": bool, "need_summary": str, "recommended_action": str}
"""

async def qualify_lead(conversation: list[dict], tenant_id: str) -> dict:
    """Analyze conversation and return lead qualification score."""
    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=512,
        system=QUALIFICATION_PROMPT,
        messages=[{"role": "user", "content": str(conversation)}]
    )
    result = json.loads(response.content[0].text)

    # Persist lead score
    async with get_db() as db:
        lead = Lead(
            tenant_id=tenant_id,
            score=result["score"],
            budget=result["budget"],
            timeline=result["timeline"],
            need_summary=result["need_summary"],
            status="hot" if result["score"] >= 70 else "warm" if result["score"] >=
40 else "cold"
        )
```

```
db.add(lead)
await db.commit()

# Notify owner if hot lead
if result["score"] >= 70:
    await notify_owner_hot_lead(lead, tenant_id)

return result
```

### 3.3 Lead Score Model (DB)

```
class Lead(Base):
    __tablename__ = "leads"
    id = Column(UUID, primary_key=True, default=uuid4)
    tenant_id = Column(UUID, ForeignKey("tenants.id"), nullable=False)
    conversation_id = Column(UUID)
    score = Column(Integer) # 0-100
    status = Column(String) # hot / warm / cold
    budget = Column(String)
    timeline = Column(String)
    need_summary = Column(Text)
    contact_name = Column(String)
    contact_email = Column(String)
    contact_phone = Column(String)
    created_at = Column(DateTime, default=func.now())
    followed_up_at = Column(DateTime)
    converted = Column(Boolean, default=False)
```

### 3.4 Integration Point in agent.py

Call `qualify_lead()` at the end of every conversation that lasts more than 3 turns:

```
# In agent.py - add at conversation close / after 3+ turns
from app.services.lead_qualifier import qualify_lead

if len(conversation_history) >= 3:
    lead_data = await qualify_lead(conversation_history, tenant_id)
    # Lead is saved to DB automatically inside qualify_lead()
```

## 4. Instant Quote / Estimate Generator

### 4.1 Overview

The bot collects service requirements through a natural conversation and generates a formatted price estimate instantly. The estimate is delivered in-chat and optionally emailed as a PDF. No more waiting days for quotes — customers get an answer immediately.

### 4.2 New File: `services/quote_generator.py`

```
from anthropic import Anthropic
from app.models.quote import Quote
from app.services.knowledge import retrieve_context
import json

client = Anthropic()

async def generate_quote(requirements: str, tenant_id: str) -> dict:
    """Generate a price estimate based on customer requirements."""
    # Pull pricing info from the tenant's knowledge base
    pricing_context = await retrieve_context(
        query=f"pricing price cost estimate {requirements}",
        tenant_id=tenant_id
    )

    prompt = f"""
    Based on this business's pricing information:
    {pricing_context}

    Customer requires: {requirements}

    Generate a professional quote. Return JSON:
    [{"line_items": [{"description": str, "price": float}]},
    "subtotal": float, "notes": str, "valid_until": str,
    "chat_summary": str}]
    """

    response = client.messages.create(
        model="claude-sonnet-4-20250514", max_tokens=1024,
        messages=[{"role": "user", "content": prompt}]
    )
    result = json.loads(response.content[0].text)

    # Save quote to DB
    await save_quote(result, tenant_id)

    return result
```

 **Setup Required**

For the quote generator to work, each tenant must upload their pricing document (PDF, XLSX, or DOCX) via the Knowledge Sources page. The bot will automatically pull from it during quote generation.

## 5. WhatsApp Proactive Follow-up & Broadcast Campaigns

### 5.1 Overview

Two related capabilities: (a) automated follow-up sequences for individual leads who chatted but did not convert, and (b) broadcast campaigns for sending personalised messages to a tenant's customer list. WhatsApp broadcasts have a 98% open rate vs 20% for email.

### 5.2 New File: `services/whatsapp_campaigns.py`

```
from app.core.whatsapp import send_whatsapp_message
from app.models.campaign import Campaign, CampaignContact
from apscheduler.schedulers.asyncio import AsyncIOScheduler

async def send_followup_sequence(lead_id: str, tenant_id: str):
    """Send time-delayed follow-up messages to a lead."""
    sequences = [
        {"delay_hours": 2, "template": "followup_1_hour"},
        {"delay_hours": 24, "template": "followup_24_hour"},
        {"delay_hours": 72, "template": "followup_3_day"},
    ]
    for seq in sequences:
        scheduler.add_job(
            send_whatsapp_message,
            "date",
            run_date=datetime.now() + timedelta(hours=seq["delay_hours"]),
            args=[lead_id, seq["template"], tenant_id]
        )

async def send_broadcast(campaign_id: str, tenant_id: str):
    """Send a personalised broadcast to all contacts in a campaign."""
    campaign = await get_campaign(campaign_id)
    contacts = await get_campaign_contacts(campaign_id)

    for contact in contacts:
        personalised = campaign.message_template.format(
            name=contact.name,
            business=contact.company
        )
        await send_whatsapp_message(
            phone=contact.phone,
            message=personalised,
            tenant_id=tenant_id
        )
        await log_campaign_send(campaign_id, contact.id)
```

### 5.3 New DB Models



```
class Campaign(Base):
    __tablename__ = "campaigns"
    id = Column(UUID, primary_key=True, default=uuid4)
    tenant_id = Column(UUID, ForeignKey("tenants.id"))
    name = Column(String)
    message_template = Column(Text) # Supports {name}, {company}, {offer}
    campaign_type = Column(String) # "broadcast" | "followup_sequence"
    status = Column(String) # "draft"|"scheduled"|"sending"|"complete"
    scheduled_at = Column(DateTime)
    created_at = Column(DateTime, default=func.now())

class CampaignContact(Base):
    __tablename__ = "campaign_contacts"
    id = Column(UUID, primary_key=True, default=uuid4)
    campaign_id = Column(UUID, ForeignKey("campaigns.id"))
    phone = Column(String)
    name = Column(String)
    sent_at = Column(DateTime)
    delivered = Column(Boolean, default=False)
    replied = Column(Boolean, default=False)
```

## 6. Post-Sale Automation Suite

### 6.1 Three Automated Flows to Build

|                                                                                                                                                                      |                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Post-Sale Check-in</b><br>3 days after a purchase/booking, bot sends a WhatsApp: "How did it go? Any issues?"<br>Catches problems early, signals professionalism. | <b>Google Review Request</b><br>After a positive sentiment check-in, bot sends a personalised Google review link. Automated but feels human. |
| <b>Referral Program</b><br>Happy customer auto-receives a personalised referral code + offer ("Refer a friend, both get 10% off"). Tracks referrals in DB.           | <b>Reactivation Campaign</b><br>Customers who haven't engaged in 60+ days auto-receive a win-back WhatsApp with a tailored offer.            |

### 6.2 New File: services/post\_sale\_automation.py

```
from app.services.sentiment import analyse_sentiment
from app.core.whatsapp import send_whatsapp_message
from app.models.customer import Customer

async def trigger_post_sale_flow(order_id: str, tenant_id: str):
    """Schedule all post-sale messages for an order."""
    scheduler.add_job(send_checkin, "date",
                      run_date=datetime.now()+timedelta(days=3),
                      args=[order_id, tenant_id])

    scheduler.add_job(send_referral_offer, "date",
                      run_date=datetime.now()+timedelta(days=7),
                      args=[order_id, tenant_id])

async def send_checkin(order_id: str, tenant_id: str):
    customer = await get_customer_for_order(order_id)
    msg = f"Hi {customer.name}! Just checking in – how did everything go? Reply here."
    await send_whatsapp_message(customer.phone, msg, tenant_id)

async def handle_checkin_reply(reply: str, customer_id: str, tenant_id: str):
    """Analyse checkin reply. If positive, request review."""
    sentiment = await analyse_sentiment(reply)
    if sentiment["label"] == "positive" and sentiment["score"] > 0.75:
        review_url = await get_google_review_url(tenant_id)
        msg = (f"So glad to hear that! Would you mind leaving us a quick Google review? "
              f"It means a lot to us: {review_url}")
        await send_whatsapp_message(customer.phone, msg, tenant_id)
    elif sentiment["label"] == "negative":
```

```
await escalate_to_human(customer_id, reply, tenant_id)
```

## 7. Conversation Analytics & Business Intelligence

### 7.1 Dashboard Metrics to Build

The analytics dashboard gives SME owners insights they have never had before. This is a strong retention and upsell driver — owners see clear ROI from the platform.

| Acquisition Metrics                                                                                   | Support Metrics                                                                 | Revenue Metrics                                                                                |
|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Total leads this week Hot / warm / cold breakdown<br>Lead source (web vs WhatsApp)<br>Conversion rate | Avg response time<br>Top 10 FAQ questions<br>Resolution rate<br>Sentiment trend | Quotes sent & accepted<br>Estimated lost revenue<br>Referrals generated<br>Campaign open rates |

### 7.2 New File: services/analytics.py

```
from sqlalchemy import func, text
from app.models.lead import Lead
from app.models.conversation import Conversation

async def get_dashboard_metrics(tenant_id: str, days: int = 7) -> dict:
    """Aggregate all KPIs for the analytics dashboard."""
    since = datetime.now() - timedelta(days=days)

    async with get_db() as db:
        leads = await db.execute(
            select(func.count(), Lead.status)
            .where(Lead.tenant_id == tenant_id)
            .where(Lead.created_at >= since)
            .group_by(Lead.status)
        )
        top_questions = await db.execute(
            text("SELECT query, count(*) AS frequency FROM knowledge_gaps "
                "WHERE tenant_id = :tid AND created_at >= :since "
                "GROUP BY query ORDER BY frequency DESC LIMIT 10"),
            {"tid": tenant_id, "since": since}
        )
        lost_value = await estimate_lost_revenue(tenant_id, since)

    return {
        "leads": {row.status: row.count for row in leads},
        "top_questions": [r._asdict() for r in top_questions],
        "estimated_lost_revenue": lost_value,
        "period_days": days
    }
```

### 7.3 Lost Opportunity Alert

A daily scheduled job calculates estimated lost revenue from unconverted hot leads and sends an alert to the tenant owner. This makes the platform's ROI tangible and drives paid upgrades.

```
# In workers/knowledge_refresh.py – add alongside existing scheduler

@scheduler.scheduled_job("cron", hour=8, minute=0)
async def daily_lost_opportunity_alert():
    tenants = await get_all_tenants()
    for tenant in tenants:
        hot_leads_unconverted = await count_unconverted_hot_leads(tenant.id)
        if hot_leads_unconverted > 0:
            avg_deal = tenant.settings.get("avg_deal_value", 500)
            lost = hot_leads_unconverted * avg_deal * 0.3 # 30% avg close rate
            msg = (f"NexusAI Daily Report: You had {hot_leads_unconverted} hot
leads "
                  f"yesterday that didn't convert. Est. missed revenue:
£{lost:.0f}.")
            await send_email(tenant.owner_email, "Daily Lead Report", msg)
```

## 8. Payment Collection Inside Chat

### 8.1 Overview

The bot generates a Stripe payment link directly inside the conversation. The customer pays without leaving WhatsApp or the website chat. Ideal for deposits, bookings, retainers, and small product purchases. Stripe is already in the NexusAI repo.

### 8.2 New File: `services/payment_link.py`

```
import stripe
from app.core.config import settings

stripe.api_key = settings.STRIPE_SECRET_KEY

async def create_payment_link(
    amount: float,
    description: str,
    customer_email: str,
    tenant_id: str
) -> str:
    """Create a Stripe payment link and return the URL."""
    product = stripe.Product.create(name=description)
    price = stripe.Price.create(
        product=product.id,
        unit_amount=int(amount * 100), # Stripe uses pence/cents
        currency="gbp",
    )
    link = stripe.PaymentLink.create(
        line_items=[{"price": price.id, "quantity": 1}],
        metadata={"tenant_id": tenant_id, "customer_email": customer_email}
    )
    return link.url

# In agent.py - detect payment intent and trigger link
# Add to tool_calling logic:
PAYMENT_TOOL = {
    "name": "send_payment_link",
    "description": "Send a payment link to the customer when they agree to pay or book",
    "input_schema": {
        "type": "object",
        "properties": {
            "amount": {"type": "number"},
            "description": {"type": "string"}
        },
        "required": ["amount", "description"]
    }
}
```



## 9. Industry Templates System

### 9.1 Overview

Pre-built templates for 6 core SME verticals. Each template contains: (a) a seed knowledge base with common FAQs for that industry, (b) a pre-configured conversation flow for that business type, (c) sample responses the AI can draw from immediately on day one.

This is your biggest sales accelerator — an SME is live in 30 minutes, not 30 days.

### 9.2 Templates to Build

| Industry         | Key Automations                                                       | Sample Seed Knowledge Topics                                            |
|------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------|
| Restaurant       | Menu Q&A, table booking, opening hours, delivery ETA, dietary queries | Full menu, allergen info, opening hours, delivery zones, special offers |
| Salon / Spa      | Appointment booking, price list, staff availability, product sales    | Service menu, pricing, staff bios, cancellation policy, gift vouchers   |
| Real Estate      | Property enquiries, viewing bookings, mortgage FAQs, area guides      | Current listings, area info, buyer/seller process, fee structure        |
| Clinic / Dentist | Appointment booking, services, insurance FAQs, emergency triage       | Services, fees, insurance accepted, aftercare guides, team bios         |
| E-commerce       | Order status, returns, product recommendations, abandoned cart        | Return policy, shipping times, product catalogue, size guides           |
| Trades           | Quote requests, availability, emergency callouts, follow-up           | Service areas, pricing guide, certifications, emergency contacts        |

### 9.3 Template Storage Structure

```
# backend/app/data/templates/
templates/
  restaurant/
    knowledge_base.json    ← seed FAQs and answers
    conversation_flow.json ← structured question paths
    system_prompt.txt      ← industry-specific agent persona
  salon/
  real_estate/
  clinic/
  ecommerce/
  trades/

# Apply template on tenant onboarding:
async def apply_industry_template(tenant_id: str, industry: str):
    template = load_template(industry)
    for faq in template["knowledge_base"]:
```



```
embedding = await get_embedding(faq["content"])
await store_in_pgvector(faq["content"], embedding, tenant_id)
```

# 10. Key Integrations

## 10.1 Priority Integration List

|                                                                                                                                                                                |                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Shopify / WooCommerce</b><br>Order status, returns, product Q&A, abandoned cart recovery. Install as a plugin app. Syncs product catalogue to knowledge base automatically. | <b>Google Calendar / Calendly</b><br>Appointment booking directly inside WhatsApp or web chat. Bot checks real-time availability and confirms instantly. |
| <b>Google Business Profile</b><br>Auto-sync business hours, address, FAQs, and services. When Google updates, knowledge base updates.                                          | <b>Zapier Webhook</b><br>Connect to any other business tool (accounting, inventory, stock). Simple outbound webhook on key events.                       |
| <b>WhatsApp Catalog</b><br>Display product/service catalogue natively inside WhatsApp. Customer browses and enquires without leaving the app.                                  | <b>Email via SendGrid (existing)</b><br>Auto-email conversation transcripts to owners, send quote PDFs to customers, trigger post-sale emails.           |

## 10.2 Shopify Integration (Highest Priority)

```
# New file: services/integrations/shopify_connector.py
import shopify

async def sync_shopify_products(tenant_id: str, shop_url: str, access_token: str):
    """Sync Shopify product catalogue to knowledge base."""
    session = shopify.Session(shop_url, "2024-01", access_token)
    shopify.ShopifyResource.activate_session(session)

    products = shopify.Product.find()
    for product in products:
        text = f"{product.title}: {product.body_html}. Price:
{product.variants[0].price}"
        embedding = await get_embedding(text)
        await store_in_pgvector(text, embedding, tenant_id, source_type="shopify")

async def get_order_status(order_number: str, tenant_id: str) -> str:
    """Called by the agent tool to look up a Shopify order."""
    shopify_creds = await get_tenant_shopify_creds(tenant_id)
    order = shopify.Order.find(order_number)
    return f"Order {order_number}: {order.fulfillment_status} -
{order.shipping_address.city}"
```

# 11. Sentiment-Based Escalation (Extend Existing)

The sentiment.py service already exists in the repo. Extend it to trigger real-time escalation when a customer becomes frustrated.

## 11.1 Escalation Logic to Add to agent.py

```
# In agent.py - check sentiment on every user message
from app.services.sentiment import analyse_sentiment

sentiment = await analyse_sentiment(user_message)

if sentiment["label"] == "negative" and sentiment["score"] > 0.80:
    # Offer immediate human escalation
    escalation_msg = (
        "I can see this is frustrating. Would you like me to connect you "
        "with one of our team members right now? They'll take over immediately."
    )
    await send_message(escalation_msg, conversation_id)
    await alert_human_agent(conversation_id, tenant_id,
reason="negative_sentiment")

    # Also notify owner via WhatsApp
    await send_whatsapp_to_owner(
        tenant_id,
        f"🚨 Unhappy customer alert: {user_message[:100]}... - Review in
dashboard."
    )
```

# 12. Frontend — New Dashboard Pages

Add the following pages to the Next.js dashboard. Use the existing Tailwind + shadcn/ui setup throughout.

| Page / Route                 | Key UI Components                                                                                                   |
|------------------------------|---------------------------------------------------------------------------------------------------------------------|
| /dashboard/leads             | Lead pipeline board (hot/warm/cold columns), lead detail drawer, conversion tracking                                |
| /dashboard/analytics         | KPI cards, conversation volume chart, top questions table, sentiment trend graph, lost revenue alert                |
| /dashboard/campaigns         | Campaign list, message template editor with {variable} support, contact CSV upload, schedule picker, delivery stats |
| /dashboard/knowledge-sources | File upload dropzone, URL crawler input, CRM connect buttons, source list with sync status, gaps log (from v1 spec) |

|                      |                                                                                                |
|----------------------|------------------------------------------------------------------------------------------------|
| /dashboard/quotes    | Quote list, preview modal, resend/email options, accepted vs pending status                    |
| /onboarding/industry | Industry template picker (6 tiles), instant preview of what the bot will know, one-click apply |

## 13. Full Implementation Roadmap

Recommended sequence across all phases. Phase 1 is from v1 spec. Phases 2–4 are new.

| Phase | Step | Task                                                                  | Files                                      | Days |
|-------|------|-----------------------------------------------------------------------|--------------------------------------------|------|
| 1     | 1–9  | Knowledge base: web crawler, doc parser, gap filler, CRM, API, worker | See v1 spec (Sections 3–8)                 | 5    |
| 2     | 10   | Lead Qualification Engine + DB model                                  | services/lead_qualifier.py                 | 1    |
| 2     | 11   | Quote Generator + integrate with agent tool calls                     | services/quote_generator.py                | 1    |
| 2     | 12   | Sentiment escalation + human handover alert                           | Extend agent.py + sentiment.py             | 0.5  |
| 2     | 13   | Payment link generation via Stripe tool                               | services/payment_link.py                   | 0.5  |
| 3     | 14   | WhatsApp follow-up sequences + broadcast campaigns                    | services/whatsapp_campaigns.py             | 2    |
| 3     | 15   | Post-sale automation: check-in, review request, referral              | services/post_sale_automation.py           | 1    |
| 3     | 16   | Industry templates (6 verticals)                                      | data/templates/*                           | 2    |
| 3     | 17   | Analytics service + daily lost revenue alert                          | services/analytics.py                      | 1    |
| 4     | 18   | Shopify / WooCommerce integration                                     | services/integrations/shopify_connector.py | 2    |
| 4     | 19   | Google Calendar / Calendly booking integration                        | services/integrations/calendar.py          | 1    |
| 4     | 20   | Frontend: Leads, Analytics, Campaigns, Quotes pages                   | frontend/src/app/dashboard/*               | 5    |
| 4     | 21   | Industry template onboarding picker                                   | frontend/src/app/onboarding/*              | 1    |

## 14. Additional Environment Variables

```
# Lead Qualification
LEAD_QUALIFICATION_MIN_TURNS=3
HOT_LEAD_SCORE_THRESHOLD=70
WARM_LEAD_SCORE_THRESHOLD=40

# WhatsApp Campaigns
WHATSAPP_DAILY_MESSAGE_LIMIT=1000
FOLLOWUP_SEQUENCE_ENABLED=true

# Post-Sale Automation
CHECKIN_DELAY_DAYS=3
REVIEW_REQUEST_MIN_SENTIMENT=0.75
REFERRAL_DISCOUNT_PERCENT=10

# Analytics
DAILY_ALERT_HOUR=8
DEFAULT_AVG_DEAL_VALUE=500

# Integrations
SHOPIFY_API_VERSION=2024-01
GOOGLE_CALENDAR_CREDENTIALS_PATH=/secrets/google_calendar.json
```

## 15. Complete File Creation & Modification List

### Files to CREATE (v2.0 additions)

- backend/app/services/lead\_qualifier.py
- backend/app/services/quote\_generator.py
- backend/app/services/whatsapp\_campaigns.py
- backend/app/services/post\_sale\_automation.py
- backend/app/services/payment\_link.py
- backend/app/services/analytics.py
- backend/app/services/integrations/shopify\_connector.py
- backend/app/services/integrations/calendar.py
- backend/app/models/lead.py
- backend/app/models/campaign.py
- backend/app/models/quote.py
- backend/app/data/templates/restaurant/ (knowledge\_base.json, conversation\_flow.json, system\_prompt.txt)
- backend/app/data/templates/salon/ (same structure)
- backend/app/data/templates/real\_estate/ (same structure)
- backend/app/data/templates/clinic/ (same structure)

- backend/app/data/templates/ecommerce/ (same structure)
- backend/app/data/templates/trades/ (same structure)
- frontend/src/app/dashboard/leads/page.tsx
- frontend/src/app/dashboard/analytics/page.tsx
- frontend/src/app/dashboard/campaigns/page.tsx
- frontend/src/app/dashboard/quotes/page.tsx
- frontend/src/app/onboarding/industry/page.tsx

## Files to MODIFY (v2.0 additions)

- backend/app/services/agent.py — add lead qualification, sentiment escalation, payment tool, quote tool
- backend/app/services/sentiment.py — extend with escalation trigger
- backend/app/workers/knowledge\_refresh.py — add daily lost revenue alert job
- backend/app/main.py — register new API routers
- backend/requirements.txt — add shopify, google-auth, pyairtable

### Definition of Done

The platform is complete when an SME in any of the 6 industries can: (1) onboard in < 30 minutes using an industry template, (2) have the bot answer customer questions from their uploaded documents and website, (3) automatically follow up with leads via WhatsApp, (4) see their lead pipeline and analytics in the dashboard, and (5) collect payment inside the chat.